

Tarea 7: PHP con BBDD

Se debe de desarrollar en **Visual Studio Code** en la **máquina virtual** donde está instalado el entorno **LAMP** (Linux, Apache, MariaDB y PHP).

Parte 1 - Conexión con BBDD

En esta nueva tarea debes mantener exactamente la misma aplicación de la Tarea 6 pero con un cambio fundamental: **los datos ya no se almacenarán en un array, sino en una base de datos MariaDB instalada en el entorno LAMP.**

Esto implica que:

- Deberás eliminar el array como sistema de almacenamiento
- Los contactos deberán insertarse, modificarse y eliminarse mediante consultas SQL
- El listado de contactos deberá cargarse leyendo los datos desde la base de datos
- La lógica de validaciones seguirá siendo exactamente la misma que en la Tarea 6

El **objetivo** de esta tarea es transformar una aplicación con almacenamiento en memoria en una aplicación con persistencia real de datos.

Previamente, se debe de **crear una base de datos llamada “contactos”** y una **tabla llamada “contactos”** con los siguientes campos:

- ID (INT, clave primaria, autoincrement)
- Nombre (VARCHAR(25))
- Email (VARCHAR(25))

La aplicación debe de estar organizada en los **siguientes archivos**:

- agenda.php (lógica de la aplicación y la parte visual)
- conexion.php (establecer la conexión con la base de datos y probar si hay errores)

Conexión con la base de datos

En este caso se deberá utilizar la **conexión mediante PDO**, implementada en el archivo **conexion.php**, creando una **función** que reciba como parámetros el nombre de la base de datos, el usuario y la contraseña, y que devuelva el objeto de conexión.

La función de conexión deberá **gestionar posibles errores utilizando una estructura try/catch**, mostrando un mensaje de error en caso de que la conexión no pueda realizarse.

La función de conexión definida en `conexion.php` deberá ser utilizada desde `agenda.php`, incluyendo previamente este archivo mediante *require* y la función incluyendo los parámetros:

```
// Conectamos con la base de datos
require "conexion.php";
$conexion = conectar("contactos", "root", "admin123");
```

Requisitos técnicos

Todas las operaciones con la base de datos deberán realizarse utilizando **consultas preparadas** (prepare/execute). (En los apuntes aparece esto únicamente vinculado a la conexión mysqli).

Las consultas preparadas permiten separar la estructura de la consulta SQL de los datos introducidos por el usuario, evitando problemas de seguridad como inyección SQL.

El funcionamiento es el siguiente:

- Se prepara la consulta utilizando **prepare()**, indicando los parámetros mediante ?
- Se ejecuta la consulta mediante **execute()**, pasando los valores correspondientes

Ejemplo:

```
$stmt = $conexion->prepare("SELECT * FROM contactos WHERE nombre = ?");
$stmt->execute([$nombre]);
$contacto = $stmt->fetch(PDO::FETCH_ASSOC);

if ($contacto) {
    echo "El contacto existe";
}
```

El método **fetch()** permite obtener una fila del resultado de la consulta.

Si no encuentra ningún resultado, devuelve false.

Utilizando `PDO::FETCH_ASSOC` los datos se devuelven en forma de array asociativo, utilizando como claves los nombres de las columnas de la tabla

Parte 2 - Inclusión de Login

Una vez realizada la conexión con la base de datos, se añadirá un **sistema de autenticación mediante sesiones** para que solo los usuarios identificados puedan acceder a la agenda. (Revisa los apuntes de la UT7).

Base de datos

Se deberá crear una **nueva tabla llamada usuarios** en la base de datos contactos, con los siguientes campos mínimos:

- id (INT, PRIMARY KEY, AUTOINCREMENT)
- email (VARCHAR(50), UNIQUE)
- password_hash (VARCHAR(255)) - hash de la contraseña

Las contraseñas deberán almacenarse utilizando la función `password_hash()` y comprobarse mediante `password_verify()`. No se permite almacenar contraseñas en texto plano.

Nota sobre el almacenamiento de contraseñas

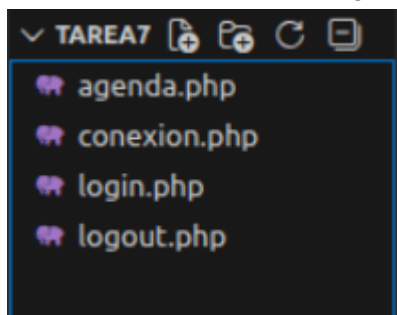
En aplicaciones reales, las contraseñas no deben almacenarse en texto plano en la base de datos. En su lugar se utiliza una **técnica llamada hash de contraseña**, que transforma la contraseña en un valor cifrado que no puede revertirse.

PHP proporciona funciones específicas para ello:

- **`password_hash(password, algoritmo)`** se utiliza para generar el hash de la contraseña que se almacenará en la base de datos.
Ejemplo: `password_hash("1234", PASSWORD_DEFAULT);`
- **`password_verify(password, hash_password)`** se utiliza para comprobar si la contraseña introducida por el usuario coincide con el hash almacenado.
Ejemplo: `password_verify($password, $usuario["password_hash"]);`

Funcionamiento y organización del proyecto

Para implementar el sistema de login será necesario añadir **nuevos archivos al proyecto**:



- **login.php** → mostrará el formulario de inicio de sesión y procesará los datos introducidos.



Login

Introduce tu email y contraseña:

Email: Contraseña:

Si el email o la contraseña están vacíos mostrará: "Introduce todos los datos".



Introduce todos los datos

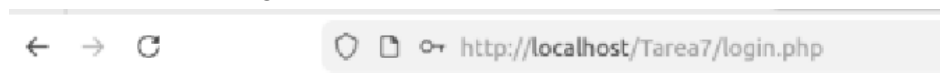
Login

Introduce tu email y contraseña:

Email: Contraseña:

Si el email aparece en la base de datos:

- Contraseña correcta: se guardarán los valores de id y email en \$_SESSION. Se redirigirá a la página de agenda.php mediante el método header. Ejemplo: header("Location: agenda.php");



Login

Introduce tu email y contraseña:

Email: Contraseña:

← → ↻ http://localhost/Tarea7/agenda.php

Agenda de contactos

[Cerrar sesión](#)

Contactos

Rebeca - rebeca@prueba.es

Nuevo contacto

Nombre: Email:

- Contraseña incorrecta. Mostrar mensaje: “Contraseña incorrecta”

← → ↻ http://localhost/Tarea7/login.php

Contraseña incorrecta

Login

Introduce tu email y contraseña:

Email: Contraseña:

Si el email no aparece en la base de datos. Mostrar mensaje: “El email no se encuentra en la base de datos”

El email no se encuentra en la base de datos

Login

Introduce tu email y contraseña:

Email: Contraseña:

- **logout.php** → deberá:
 - Iniciar la sesión
 - Destruirla
 - Redirigir al usuario a login.php

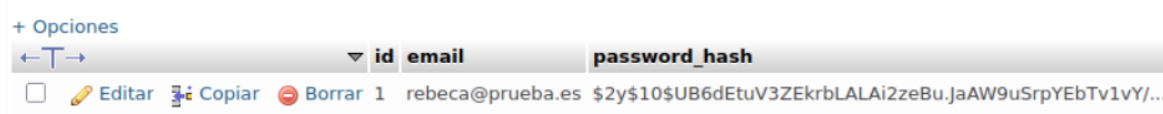
- **agenda.php** → Deberá de incluir `session_start()` y crear un botón de “Cerrar sesión” el cual se generará una llamada a `logout.php` mediante un enlace:



The screenshot shows a web browser address bar with the URL `http://localhost/Tarea7/agenda.php`. The page title is "Agenda de contactos". Below the title, there is a button labeled "Cerrar sesión" which is highlighted with a red rectangle. Underneath, the text "Contactos" is displayed, followed by "Rebeca - rebeca@prueba.es". Below this, the heading "Nuevo contacto" is shown. At the bottom, there is a form with two input fields: "Nombre:" and "Email:", followed by an "Enviar" button.

- **conexion.php** → se seguirá utilizando para la conexión PDO (no cambiar nada)

Nota: En esta práctica, por simplificación, **no será necesario implementar una página para registrar nuevos usuarios**. Los usuarios se crearán previamente de forma manual en la base de datos para poder probar el sistema de login. Ejemplo:



The screenshot shows a table in a database management system. The table has three columns: "id", "email", and "password_hash". There is one row of data with the email "rebeca@prueba.es" and a long alphanumeric hash. The table is titled "usuarios" and has a "password_hash" column.

	id	email	password_hash
1		rebeca@prueba.es	\$2y\$10\$UB6dEtuV3ZEkrbLALAI2zeBu.JaAW9uSrpYEBTv1vY/...

En mi caso he obtenido el hash de la contraseña: 1234

Para ello se deberá:

- Generar previamente el `password_hash` mediante un pequeño script PHP con la función `password_hash()`
- Insertar el usuario manualmente en la tabla `usuarios` mediante phpMyAdmin o mediante una consulta SQL

Entrega

La tarea se debe de entregar mediante un **fichero .zip** (*apellido1_apellido2_nombre_Tarea7.zip*) que contenga:

- ❖ Todos los **archivos PHP del proyecto** (agenda.php, conexion.php, login.php, logout.php).
- ❖ Un **vídeo explicativo del funcionamiento** (duración recomendada: 3–5 minutos).
- ❖ Un **documento PDF** que incluya:
 - Breve explicación del funcionamiento de la aplicación
 - Explicación de la conexión mediante PDO
 - Plan de pruebas realizado
 - Capturas de la base de datos (estructura de tablas)

Criterios de puntuación (10 puntos)

Parte 1 – Persistencia de datos con base de datos (6 puntos)

- ❖ Conexión correcta a la base de datos mediante PDO y organización del proyecto en los archivos indicados. (2 puntos)
- ❖ Implementación del CRUD mediante consultas SQL (insertar, actualizar y eliminar contactos). (2 puntos)
- ❖ Visualización correcta del listado de contactos obtenido desde la base de datos. (1 punto)
- ❖ Formulario y procesamiento de datos correctamente implementado (validaciones incluidas). (1 punto)

Parte 2 – Sistema de autenticación y control de sesión (4 puntos)

- ❖ Implementación del sistema de login consultando los usuarios almacenados en la base de datos. (2 puntos)
- ❖ Uso correcto de password_hash() y password_verify() para el almacenamiento y verificación de contraseñas. (1 punto)
- ❖ Implementación del control de sesión y cierre de sesión (session_start, protección de agenda.php y logout). (1 punto)